

Document

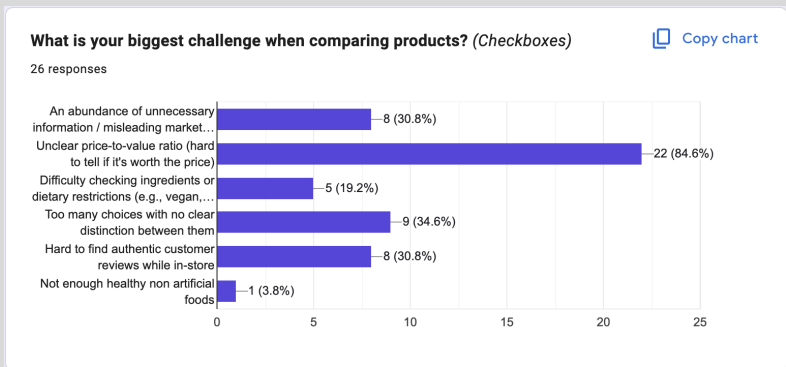
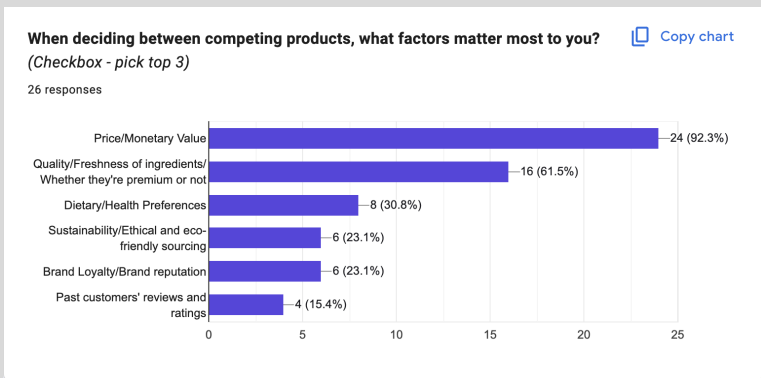
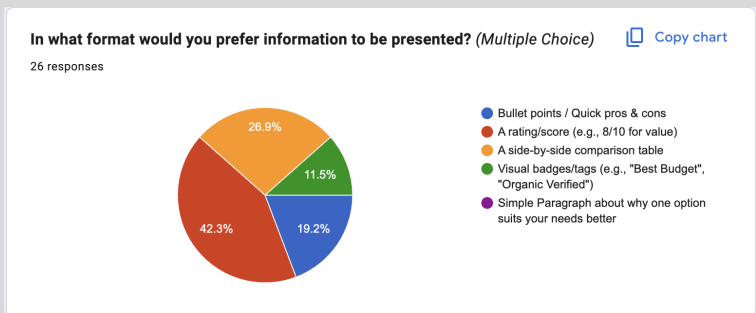
Task 1: Identify and document an opportunity for an innovative solution

Modern-day consumers are often presented with a multitude of products claiming to be unique from their competition, but fail to identify what makes them so special. This oversaturation of content within supermarkets and retail chains make it difficult for consumers to understand what fits their needs – whether if it's their preferences or getting the most bang for their buck. A potential approach to simplifying the incoherent abundance of unnecessary information provided by retailers involves **developing an innovative application that consolidates consumer-facing information into what suits their needs.**

Task 2: Develop a Gantt Chart

 VCE Applied Computing Unit 2 Outcome 1 – Patrick & Safin

Task 3: Conduct surveys and interviews to determine solution requirements



Open Feedback: What is one feature or capability that would make this app indispensable for you? (Paragraph / Short Answer)

26 responses

- No sign up, No fees or free trials. a free quick and easy to use app will be the best
- Reduced notifications or alerts.
- Ability to see the spending trends of people you know
- The inclusion of allowing me to barcode scan military aircraft. Thank you.
- IF it didn't include ads, gave me a simplified breakdown on the qualities such as: Calories, protein, cost compared to competitors
- an AI shopping assistant that summarises a products cons, pros, reviews etc. and is able to suggest products according to personalised preferences
- Have a family mode where there can be a list for food shopping so parents can remember what their child would like

Task 4: Document evaluation criteria

Criteria ID	Evaluation criteria	Testing metric	Relevance to goals
1	Accuracy of eco-score	Select two common products in the consumer market, and manually verify that the score provided by the solution is calculated accurately	This ensures that the solution created effectively combats the confusing and generic greenwashing terms without misrepresenting data
2	Accuracy of	Input a price and a weight,	This ensures that the solution presents

	cost-effectiveness score	verifying that the price per unit score provided by the solution is calculated accurately	information for consumers to apply to get high value-for-money calculations
3	Processing speed	Interact with an element of the solution and ensure the results appear on the screen within one second	This ensures that the solution is fast enough to be applied practically in a real-world shopping environment
4	Input error resilience	Input letters into a numerical value only input box and ensure that the solution not only does not crash but also explains the issue to the user	This ensures that the solution holds high robustness and user-friendliness when mistakes inevitably occur
5	Interface reliability	Check that low scores are recorded in red, medium in orange and high in green, and ensure that all text can be read from a distance of 50cm away	This ensures that the solution can be easily interpreted by a user, increasing ease of use
6	Solution portability	Transfer the final .py script to a clean, default school computer. Run it to confirm it executes instantly without erroring	This ensures that the solution adheres to a variety of hardware environments, allowing for a variety of end-users

Task 5: Identify emerging trends

A major trend emerging globally in digital consumption is the gradual yet prominent shift toward eco-conscious tech systems. Consumers are making an attempt to decrease their environmental footprints, but they face information overload and deceptive marketing, commonly known as "greenwashing". Current software trends focus on data transparency, taking vast and confusing manufacturing datasets and translating them into simple metrics for everyday users. Our solution directly aligns with this trend by automating the environmental evaluation process at the local shelf level.

Instead of forcing users to read text labels, our solution's core function acts as a processing engine. It parses complex qualitative data variables, such as manufacturing locations, crop types, and packaging elements, and transforms them into a structured, unified numerical value: an eco-score. This eco-score generalises itself across products, removing the incoherent abundance of unnecessary information into one uniform evaluation of the product.

Task 6: Identify legal, economical and ethical considerations

Under intellectual property and consumer laws, using real supermarket product names and brand logos within our solution without permission *can* be seen as deceptive or infringing on intellectual property. To remain fully compliant, our solution and its associated database will need to comply with individual retail regulations, which must be researched prior to product launch.

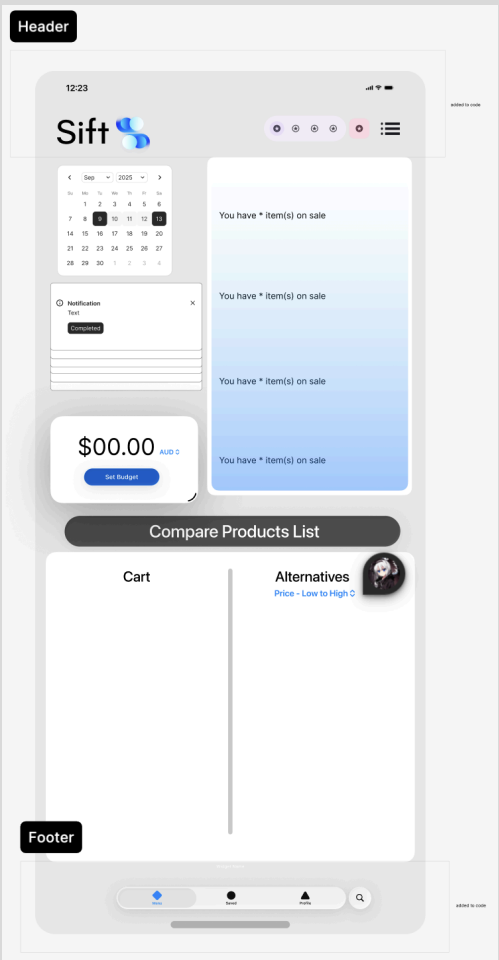
Formulas used in the eco-score function are inherently subjective and trained on large datasets that may not account for all product details. For example, deciding that local transportation practices are worth 4 points while organic farming is worth 3 points reflects an algorithmic bias that might unfairly penalize certain global but highly sustainable producers. To maintain complete transparency, we will document our scoring system inside the software interface. Users will be able to clearly see exactly how the 10-point scale is broken down, preventing bias.

Economically, eco-friendly products often carry a higher retail price tag. During periods of high inflation and cost-of-living pressures, purely focusing on environmental scores could alienate lower-income shoppers. The application directly solves this economic dilemma by processing price-per-unit calculations, empowering the user to balance their environmental values directly against their personal household budget limitations.

Task 7: Design the solution

The design draft was created using a well-known design software named Figma, used typically for corporate and entrepreneurial applications and design innovations.

The design was only a rough guide for the innovative solution created by us, where many features within the design were waived, omitted and modified to better fit the functional requirements we have prioritised for the solution.



Task 8: Develop a proof of concept

Python File, main project.

Task 9: Document functions and techniques

Function ID	Function Name	Function Details
1	Realistic iOS Layout with dynamic time and battery information from host device	Displays real-time information of the system similarly to how the hotbar of the laptop itself displays it. Real-time battery percentage that increases and decreases with charge or lack of, real time display of time in accordance to the time zone selected and displayed on the device.
2	Appearance & Theme Selection for Accessibility	User interacts with the theme dropdown menu present on all tabs of the application, changing the theme of the application to suit their needs, including a dark mode, mono-duo-trichromia and inverted mode.
3	Profile Creation, Details and Sign-in-Sign-out functionality with cached data for restart persistence	Button to open up a section dedicated to accounts. Upon first launch, the user is presented with an automatically generated guest account. They are provided with an option to create an account that will be saved upon launch, they can also choose to log out of this account if need be.

4	Product Database Search, inclusive of retailers, health information, and price	Hard coded products are inserted into the database. The hardcoded products can be searched for via the search icon, this will display its name, quantity, price, quality and varying store options to help identify better prices, quality or simply unique purchasable options based on location and other aspects.
5	Product-to-Cart Addition system complete with retailer information and price for user convenience	Upon finding their desired product, the user is able to add it to their cart utilising a set button, one for each store it is available to provide versatility and diversification amongst the products varying retailers. Once in the cart, on the menu screen, the user is able to see the product alongside its store and price located within the cart section of the menu UI.
6	Functional Page Navigation Bar	
7	Google Maps Launcher Button	Near the bottom of the menu ui, there are three buttons displayed. One of these buttons, displaying the icon of a pin from google maps, allows for the user to select it and be taken to the navigation section of the application. Within this section, the app tells the user the four stores that will be shown around their location and provides a button that will launch google maps, automatically pinpointing the nearby four retailers, central to the user's location.
8	Barcode Scanner with API-fetched health information	Upon selecting the search icon, the user is presented with a camera icon as an alternative to the traditional search. Once this alternative is selected, the camera application is launched and from there, the user can present the barcode of, specifically, a grocery based item to find out important information, such as its presented name and health star rating.
9	Portability	Discuss how Python is universal across Windows, Linux and MacOS applications, and how things such as data files and image resources were either shifted online or into the code to reduce dependencies on external sources.
10	Persistent Settings	The user interacts with the settings button on the menu ui to open this. Within this settings section of the app, the controls for the persistent settings are located. The user can set themes as a base or a set theme upon launch, this removes the need to constantly alter the theme to the user' preference when relaunching the app. They are still able to change the theme outside of this section but it will be session based and not saved as their preference unless it is selected within the settings menu. The same is said for the sort by relevance option.

Task 10 & 11: Document testing methodology & execute tests

Test ID	Test Name	Test Details	Test Execution
T-0	Themes Selection Open	Test the opening and closing of the horizontal themes dropdown bar to ensure that animations and buttons slide in and out correctly.	Unsuccessful. The dropdown bar is bunched up, making it look very unattractive and unfunctional.
T-1	Themes Selection Open	Test the opening and closing of the horizontal themes dropdown bar to ensure that animations and buttons slide in and out correctly.	Successful. The code was fixed following T-0 to ensure that the dropdown bar opens and closes correctly.

T-2	Themes Selection Interaction	Test the functionality of interacting with specific theme buttons within the theme dropdown menu, such as light mode, dark mode, mono-duo-trichromatic mode, and inverted mode.	Unsuccessful. Some elements within the application were coloured incorrectly, changing to the wrong colour, or not changing at all due to not being bound.
T-3	Themes Selection Interaction	Test the functionality of interacting with specific theme buttons within the theme dropdown menu, such as light mode, dark mode, mono-duo-trichromatic mode, and inverted mode.	Successful. The code was fixed following T-2 to ensure that all elements within the application function as intended, changing to the correct colour.
P-0	Profile Menu Comprehensive	Test the functionality of the profile menu, including opening and closing, signing in, signing out, persistence, and visualisation.	Successful. All features functioned as intended.
N-0	Navigation Bar Interaction	Test the functionality of the navigation bar, including changing between pages.	Unsuccessful. Elements from the MAIN page were spilling into the MAP and SETTINGS pages due to not being bound correctly.
N-1	Navigation Bar Interaction	Test the functionality of the navigation bar, including changing between pages.	Unsuccessful. Issues were not correctly resolved, and persisted after troubleshooting. Further refinement will be necessary.
N-2	Navigation Bar Interaction	Test the functionality of the navigation bar, including changing between pages.	Successful. The code was fixed following N-0 and N-1 to ensure that elements within the application did not spill into other pages, loaded pages correctly, and animations functioned smoothly in an attractive and realistic manner.
S-0	Search Button Open	Test the opening and closing of the search overlay engine after interaction with the search button present within the navigation bar.	Unsuccessful. Issues were derived from element layering issues, where the search overlay engine should have been the top-most elements on the page.
S-1	Search Button Open	Test the opening and closing of the search overlay engine after interaction with the search button present within the navigation bar.	Successful. The code was fixed following S-0 to ensure that elements were "lifted" and placed on the top-most layer of the application. This logic was then expanded to <i>all</i> elements within the application to improve reliability.
S-2	Search Overlay Interaction	Test the interaction with the text input box within the search overlay engine, ensuring that results are displayed correctly on the search results modal.	Unsuccessful. Text input logic functioned as intended, but the fuzzy-search logic imported by library difflab was not sensitive enough, and was too lenient.
S-3	Search Results	Test the functionality of the search results modal,	Unsuccessful. Fuzzy-search logic

	Interaction	ensuring that results are displayed correctly.	was still too lenient. A two-priority system was implemented to ensure that the correct results were prioritised.
S-4	Search Results Interaction	Test the functionality of the search results modal, ensuring that results are displayed correctly.	Successful. The code was fixed following S-2 and S-3 to ensure that the correct items were presented on the search results modal.
S-5	Search Results Interaction	Test the functionality of item-specific details such as health rating, price, and retailer information.	Successful. Information was presented on the user-facing UI as intended.
S-6	Search Addition Interaction	Test the functionality of the add to cart button on the search results modal, ensuring it correctly closes the search results modal and displays the item in the users cart within the main page.	Unsuccessful. Search results modal closing logic function as intended, but the information was incorrectly displayed on the page.
S-7	Search Addition Interaction	Test the functionality of the add to cart button on the search results modal, ensuring it correctly closes the search results modal and displays the item in the users cart within the main page.	Successful. The code was fixed following S-6 to ensure that the information was presented correctly on the main page.
S-8	Search Addition Interaction with Navigation Bar Interaction	Test the functionality of the add to cart button on the search results modal while changing pages through the navigation bar to ensure that elements are correctly added to the main page, and not others.	Unsuccessful. This issue was found by accident, and occurred as a result of elements being added to the global canvas rather than the page-specific canvas.
S-9	Search Addition Interaction with Navigation Bar Interaction	Test the functionality of the add to cart button on the search results modal while changing pages through the navigation bar to ensure that elements are correctly added to the main page, and not others.	Successful. The code was fixed following S-8 to ensure that the cart item label remained on the main page and did not spill into other pages.
B-0	API Endpoint Testing	Test the functionality of the API calling system through the imported library requests to ensure the application can successfully interact with the database the program functions off of.	Unsuccessful. Debug statements indicated that the database returned a status code of 403, which is Forbidden.
B-1	API Endpoint Testing	Test the functionality of the API calling system through the imported library requests to ensure the application can successfully interact with the database the program functions off of.	Successful. The code was fixed following B-0 by adding a specific application-targeted User-Agent that the database requires through referencing the API documentation.
B-2	Barcode Detection Testing	Test the functionality of the barcode scanning system through the imported libraries open-cv and pyzbar to ensure the application can access the user's camera and accurately fetch a barcode brought into frame.	Successful. The application successfully accessed the user's camera and detected a barcode present within the FOV of the device's camera.
B-3	Barcode Detection Testing with API Integration	Test the functionality of the barcode scanning system to ensure it can accurately pass barcode information to the API, which fetches nutritional information and stores it within the applications memory.	Successful. The application successfully detected the barcode, passed information to the API, received information

			from the API, and returned it to the user, presenting it on their UI and adding it as a new cart item. The API can only fetch information about food products. This is a known limitation of the application, and has been acknowledged by the developers.
--	--	--	--

Project Journal

24/07/2026

Code development and design development

This day marked the beginning of code development, being made alongside the progression of our UI concept design, implementing those features as we go. This day involved creating the core Apple-style UI to create an immersive and attractive appearance, by applying the TKinter library. With prior knowledge and reference to documentation, the foundation of the application was built successfully.

27/07/2026

Code development and design development

On this day, we considered the introduction of accessibility friendly features such as colourblind modes and adjustable appearances for the application. Patrick suggested we implement a functional dropdown menu for UI themes rather than hiding it in the proposed settings menu, which Safin implemented into the code over the next couple of weeks.

28/07/2026 to 31/07/2026

Code development and design development

Tasks for this period included integrating a functional page selection menu, profile log-ins and associated logic, and assistive buttons to redirect users to relevant Google Maps references. Cosmetic details such as time and battery percentage were implemented to reflect a realistic user experience. Many issues arose during this period of development, including transitioning between interface display colours and themes resulting in layout distortion due to hardcoded colour schemes. Such issues were resolved, letting the user experience be smooth and realistic.

3/08/2026 to 10/08/2026

Code development

During this period, development involved creation and perfection of the search feature. This involved creating multiple user-interactable input modals, a series of complicated fuzzy match systems through referencing documentation, and creation of the respective UI elements. It also involved the creation of a barcode scanning system as an alternative to the search method, which allows end users to scan product barcodes and fetch the appropriate health and nutritional information. Issues regarding UI elements leaking over into other page tabs were resolved during this period.

10/08/2026 to 24/08/2026

Code finalisation

All features within the Sift application were refined and finalised to prepare them for the final submission. This includes the addition of final data logic, such as the presentation of nutritional information, as well as populating the Settings page with functional legal notices, preference settings and the device's notifications manager through the use of dynamic subprocess scripts. At this stage, the application is ready for submission.

Evaluation Report

Name: Sift

Summary: a functional grocery price comparison and health rating application, used to consolidate grocery prices and health ratings from various Australian supermarkets into a single user-friendly interface

Introduction:

Sift was created as part of the Unit 2 Area of Study 1 SAC, with its name symbolic to the way it *sifts* through minefields of redundant information. Sift aims to provide Australian consumers with the resources they need to make accurate and informed decisions on products that they purchase from major retailers, without being drowned in a plethora of obfuscated terminology and greenwashing.

Discussion:

As a program, Sift does exactly what it's supposed to do: provide the user with a friendly interface to compare and gain information regarding Australian food products in a consolidated location. Features such as accessibility themes, preference management and realistic user interfaces replicate how the application would function in reality, adhering to software development expectations and guidelines.

Working off of a rough design draft, the developers – Patrick and Safin – had executive liberty over what features should be implemented, and what should otherwise be scrapped. The majority of features present within Sift were debated and discussed vigorously to determine whether it meets the solution requirements set after conducting field surveys, while aiming to meet the evaluation criteria of the application.

The developers decided that using the Python language would be the most accessible for end-users, where Python has the capability to function without issues on Windows, MacOS and Linux operating systems, reflecting the intention for the application to be universal to all Australian consumers.

As part of the functional requirements of the Unit 2 Area of Study 1 SAC, our innovative solution was required to have a physical attribute. Fitting seamlessly within our application, a key feature involves the inclusion of a barcode scanning system through the use of the user's in-built or external camera. The application fetches barcodes from the frame of view, and fetches the relevant data via an open source API. Camera frames themselves are not stored within memory, maintaining user privacy and risk of unauthorised access to cameras.